

JavaScript Key Learnings — Question 9

OJT Assessment Prep — setInterval, clearInterval, Memory Leaks & Garbage Collection

1. setInterval vs setTimeout

Both are timers — but they behave differently:

	setTimeout	setInterval
Runs	ONCE after delay	REPEATEDLY every delay ms
Stops	Automatically after running	Never — must call clearInterval
Use for	One-time delayed action	Live updates, clocks, tickers

```
setInterval(function() {  
  console.log("tick!");  
}, 1000);  
// prints "tick!" every second, forever until stopped
```

2. Storing the Interval ID — Critical Step

setInterval returns an ID number — like a ticket. You **MUST** save it to stop the interval later.

■ **Wrong — ID not saved, can never stop it:**

```
setInterval(function() {  
  updateTicker();  
}, 1000);  
// no way to stop this – memory leak!
```

■ **Correct — ID saved on this:**

```
this.intervalId = setInterval(function() {  
  updateTicker();  
}, 1000);  
// can now stop with: clearInterval(this.intervalId)
```

3. clearInterval — Stopping the Timer

clearInterval(id) permanently stops a running interval. Without it, the timer runs forever — even after the widget is removed.

```
const id = setInterval(fn, 1000); // starts repeating  
clearInterval(id);                // stops it completely  
  
// In the class:  
destroy() {  
  clearInterval(this.intervalId); // must pass the saved ID  
}
```

4. Memory Leaks — What They Are

A **memory leak** happens when your code holds onto things it no longer needs — the browser cannot free that memory.

```
// What causes the leak in this question:

// 1. Interval still running after widget closed
//    → holds reference to this.tickerNode
//    → browser cannot garbage collect it

// 2. this.tickerNode still pointing to removed element
//    → even after .remove(), memory is not freed
//    → need to set to null!
```

Over time, leaks accumulate → browser slows down → eventually crashes. This is why the question mentions users reporting crashes!

5. Garbage Collection & Setting to null

The browser's **Garbage Collector** frees memory automatically — but ONLY when nothing is pointing to an object.

■ **Wrong — element removed from DOM but still referenced:**

```
this.tickerNode.remove(); // removed from page ■
// BUT this.tickerNode still points to it in memory!
// Garbage Collector cannot free it → leak!
```

■ **Correct — set to null to break the reference:**

```
this.tickerNode.remove(); // remove from DOM
this.tickerNode = null;    // break the reference
// Nothing points to it → GC can free the memory ■
```

6. Math.random() — Generating Random Numbers

```
Math.random()           // → 0.0 to 0.999...
Math.random() * 100      // → 0.0 to 99.9...
(Math.random() * 100).toFixed(2) // → "92.88" (2 decimal places)

// For the stock ticker:
"Stock Price: $" + (Math.random() * 100).toFixed(2)
// → "Stock Price: $92.88"
```

7. The destroy() Method — 3 Required Steps

Every step is essential — missing any one causes a leak:

Step	Code	What it fixes
1. Stop timer	<code>clearInterval(this.intervalId)</code>	Stops CPU usage

2. Remove from DOM	<code>this.tickerNode.remove()</code>	Removes element from page
3. Free memory	<code>this.tickerNode = null</code>	Lets GC collect the element

■ **All 3 steps together = clean memory, no leaks, no crashes!**

■■ Complete Solution

```
class LiveTicker {
  constructor(containerId) {
    // 1. Get container
    let container = document.getElementById(containerId);

    // 2. Create and store span
    this.tickerNode = document.createElement("span");
    container.appendChild(this.tickerNode);

    // 3. Start interval – store the ID!
    this.intervalId = setInterval(() => {
      this.tickerNode.innerText =
        "Stock Price: $" + (Math.random() * 100).toFixed(2);
    }, 1000);
  }

  destroy() {
    clearInterval(this.intervalId); // stop CPU usage
    this.tickerNode.remove();       // remove from DOM
    this.tickerNode = null;         // free memory
  }
}

// Instantiate
const ticker = new LiveTicker("ticker-container");

// Wire up button
document.getElementById("stopBtn")
  .addEventListener("click", function() {
    ticker.destroy();
  });
```

■ Golden Rules to Remember

Rule	Remember
<code>setInterval</code>	Always store the returned ID — you cannot stop it without it
<code>clearInterval</code>	Always call in cleanup/destroy — missing it = CPU leak
<code>.remove()</code>	Removes element from DOM but NOT from memory
<code>= null</code>	Breaks reference so Garbage Collector can free memory

destroy()	Always implement cleanup: clearInterval + remove + null
Arrow function	Use () => in setInterval to keep "this" pointing to class